Problem A. Stair, Peak, or Neither?

Time Limit 1000 ms
Mem Limit 262144 kB

You are given three digits a, b, and c. Determine whether they form a stair, a peak, or neither.

- A *stair* satisfies the condition $a < b < c$.
- A *peak* satisfies the condition $a < b > c$.

Input

The first line contains a single integer t ($1 \le t \le 1000$) — the number of test cases.

The only line of each test case contains three digits a, b, c ($0 \le a, b, c \le 9$).

Output

For each test case, output "STAIR" if the digits form a stair, "PEAK" if the digits form a peak, and "NONE" otherwise (output the strings without quotes).

Examples

Input	Output
7 1 2 3 3 2 1 1 5 3 3 4 1 0 0 0 4 1 7 4 5 7	STAIR NONE PEAK PEAK NONE NONE STAIR

Problem B. Can I Square?

Time Limit 1000 ms
Mem Limit 262144 kB

Calin has n buckets, the i-th of which contains a_i wooden squares of side length 1.

Can Calin build a square using **all** the given squares?

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 2 \cdot 10^5$) — the number of buckets.

The second line of each test case contains n integers $a_1, \ldots, a_n$ ($1 \le a_i \le 10^9$) — the number of squares in each bucket.

The sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output "YES" if Calin can build a square using **all** of the given 1×1 squares, and "NO" otherwise.

You can output the answer in any case (for example, the strings "yEs", "yes", "Yes" and "YES" will be recognized as a positive answer).

Examples

Input	Output
5 1 9 2 14 2 7 1 2 3 4 5 6 7 6 1 3 5 7 9 11 4 2 2 2 2	YES YES NO YES NO

Note

In the first test case, Calin can build a 3×3 square.

In the second test case, Calin can build a 4×4 square.

In the third test case, Calin cannot build a square using all the given squares.

Problem C. Bark to Unlock

Time Limit 2000 ms
Mem Limit 262144 kB

As technologies develop, manufacturers are making the process of unlocking a phone as user-friendly as possible. To unlock its new phone, Arkady's pet dog Mu-mu has to bark the password once. The phone represents a password as a string of two lowercase English letters.

Mu-mu's enemy Kashtanka wants to unlock Mu-mu's phone to steal some sensible information, but it can only bark n distinct words, each of which can be represented as a string of two lowercase English letters. Kashtanka wants to bark several words (not necessarily distinct) one after another to pronounce a string containing the password as a substring. Tell if it's possible to unlock the phone in this way, or not.

Input

The first line contains two lowercase English letters — the password on the phone.

The second line contains single integer n ($1 \le n \le 100$) — the number of words Kashtanka knows.

The next n lines contain two lowercase English letters each, representing the words Kashtanka knows. The words are guaranteed to be distinct.

Output

Print "YES" if Kashtanka can bark several words in a line forming a string containing the password, and "NO" otherwise.

You can print each letter in arbitrary case (upper or lower).

Examples

Input	Output
ya 4 ah oy to ha	YES

Input	Output
hp 2 ht tp	NO

Input	Output
ah 1 ha	YES

Note

In the first example the password is "ya", and Kashtanka can bark "oy" and then "ah", and then "ha" to form the string "oyahha" which contains the password. So, the answer is "YES".

In the second example Kashtanka can't produce a string containing password as a substring. Note that it can bark "ht" and then "tp" producing "http", but it doesn't contain the password "hp" as a substring.

In the third example the string "hahahaha" contains "ah" as a substring.

Problem D. Range Minimize

Time Limit	1000 ms
Code Length Limit	50000 B
OS	Linux

You are given an array A containing N integers.
You can delete **at most two** of its elements.

Find the minimum possible value of $(\max(A) - \min(A))$ (in other words, the range of A) after the deletions.

Input Format

- The first line of input will contain a single integer T, denoting the number of test cases.
- Each test case consists of two lines of input.
  - The first line of each test case contains a single integer N — the length of the array.
  - The second line contains N space-separated integers $A_1, A_2, \ldots, A_N$.

Output Format

For each test case, output on a new line the minimum possible value of $\max(A) - \min(A)$ after at most two deletions.

Constraints

- $1 \leq T \leq 10^5$
- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq A_i \leq 10^9$
- The sum of N over all test cases won't exceed $2 \cdot 10^5$.

Sample 1

Input	Output
3 3 2 3 1 5 1 10000 10 100 1000 6 64 11 998 1005 843 945	0 99 162

Test case 1: Delete 1 and 2 to make the array $A = [3]$. $\max(A) - \min(A) = 0$ which is the best we can do.

Test case 2: Delete $A_2 = 10000$ and $A_5 = 1000$ to obtain $A = [1, 10, 100]$, for which $\max(A) - \min(A) = 99$.

Test case 3: Delete $A_1 = 64$ and $A_2 = 11$ to obtain $A = [998, 1005, 843, 945]$, which has a range of 162.

Problem E. Odd Divisor

Time Limit 2000 ms
Mem Limit 262144 kB

You are given an integer n. Check if n has an **odd** divisor, greater than one (does there exist such a number x ($x > 1$) that n is divisible by x and x is odd).

For example, if $n = 6$, then there is $x = 3$. If $n = 4$, then such a number does not exist.

Input

The first line contains one integer t ($1 \le t \le 10^4$) — the number of test cases. Then t test cases follow.

Each test case contains one integer n ($2 \le n \le 10^{14}$).

Please note, that the input for some test cases won't fit into 32-bit integer type, so you should use at least 64-bit integer type in your programming language.

Output

For each test case, output on a separate line:

- "YES" if n has an **odd** divisor, greater than one;
- "NO" otherwise.

You can output "YES" and "NO" in any case (for example, the strings yEs, yes, Yes and YES will be recognized as positive).

Examples

Input	Output
6 2 3 4 5 998244353 1099511627776	NO YES NO YES YES NO

Problem F. Matching Numbers

Time Limit 1000 ms
Mem Limit 262144 kB

You are given an integer n. Pair the integers 1 to $2n$ (i.e. each integer should be in exactly one pair) so that each sum of matched pairs is consecutive and distinct.

Formally, let (a_i, b_i) be the pairs that you matched. $\{a_1, b_1, a_2, b_2, \ldots, a_n, b_n\}$ should be a permutation of $\{1, 2, \ldots, 2n\}$. Let the sorted list of $\{a_1 + b_1, a_2 + b_2, \ldots, a_n + b_n\}$ be $s_1 < s_2 < \ldots < s_n$. We must have $s_{i+1} - s_i = 1$ for $1 \leq i \leq n - 1$.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 500$). The description of the test cases follows.

For each test case, a single integer n ($1 \leq n \leq 10^5$) is given.

It is guaranteed that the sum of n over all test cases doesn't exceed 10^5.

Output

For each test case, if it is impossible to make such a pairing, print "No".

Otherwise, print "Yes" followed by n lines.

At each line, print two integers that are paired.

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

If there are multiple solutions, print any.

Examples

Input	Output
4 1 2 3 4	Yes 1 2 No Yes 1 6 3 5 4 2 No

Note

For the third test case, each integer from 1 to 6 appears once. The sums of matched pairs are $4 + 2 = 6$, $1 + 6 = 7$, and $3 + 5 = 8$, which are consecutive and distinct.

Problem G. Array and GCD

Time Limit 2000 ms
Mem Limit 524288 kB

You are given an integer array a of size n.

You can perform the following operations any number of times (possibly, zero):

- pay one coin and increase any element of the array by 1 (you must have at least 1 coin to perform this operation);
- gain one coin and decrease any element of the array by 1.

Let's say that an array is *ideal* if both of the following conditions hold:

- each element of the array is at least 2;
- for each pair of indices i and j ($1 \le i, j \le n$; $i \neq j$) the greatest common divisor (GCD) of a_i and a_j is equal to 1. If the array has less than 2 elements, this condition is automatically satisfied.

Let's say that an array is *beautiful* if it can be transformed into an ideal array using the aforementioned operations, provided that you initially have no coins. If the array is already ideal, then it is also beautiful.

The given array is not necessarily beautiful or ideal. You can remove any elements from it (including removing the entire array or not removing anything at all). Your task is to calculate the minimum number of elements you have to remove (possibly, zero) from the array a to make it **beautiful**.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 4 \cdot 10^5$).

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($2 \le a_i \le 10^9$).

Additional constraint on the input: the sum of n over all test cases doesn't exceed $4 \cdot 10^5$.

Output

For each test case, print a single integer — the minimum number of elements you have to remove (possibly, zero) from the array a to make it **beautiful**.

Examples

Input	Output
5 3 5 5 5 4 2 3 2 4 1 3 3 2 100 2 5 2 4 2 11 2	0 2 0 0 1

Note

In the first example, you don't need to delete any elements, because the array is already beautiful. It can be transformed into an ideal array as follows: $[5, 5, 5] \to [4, 5, 5] \to [4, 4, 5] \to [4, 3, 5]$ (you end up with 3 coins).

In the second example, you need to remove 2 elements so that the array becomes beautiful. If you leave the elements $[2, 3]$ and delete the other elements, then the given array is already ideal (and therefore, beautiful).

In the third example, you don't need to delete any elements because the array is already ideal (and thus, beautiful).

In the fourth example, the array is beautiful. It can be transformed into an ideal array as follows: $[2, 100, 2] \to [2, 99, 2] \to [2, 99, 3] \to [2, 98, 3] \to [2, 97, 3]$ (you end up with 2 coins).

Problem H. Data Structures Fan

Time Limit 2000 ms
Mem Limit 262144 kB

You are given an array of integers $a_1, a_2, \ldots, a_n$, as well as a binary string[†] s consisting of n characters.

Augustin is a big fan of data structures. Therefore, he asked you to implement a data structure that can answer q queries. There are two types of queries:

- "1 l r" ($1 \le l \le r \le n$) — replace each character s_i for $l \le i \le r$ with its opposite. That is, replace all 0 with 1 and all 1 with 0.
- "2 g" ($g \in \{0, 1\}$) — calculate the value of the bitwise XOR of the numbers a_i for all indices i such that $s_i = g$. Note that the XOR of an empty set of numbers is considered to be equal to 0.

Please help Augustin to answer all the queries!

For example, if $n = 4$, $a = [1, 2, 3, 6]$, $s = \texttt{1001}$, consider the following series of queries:

1. "2 0" — we are interested in the indices i for which $s_i = \texttt{0}$, since $s = \texttt{1001}$, these are the indices 2 and 3, so the answer to the query will be $a_2 \oplus a_3 = 2 \oplus 3 = 1$.
2. "1 1 3" — we need to replace the characters s_1, s_2, s_3 with their opposites, so before the query $s = \texttt{1001}$, and after the query: $s = \texttt{0111}$.
3. "2 1" — we are interested in the indices i for which $s_i = \texttt{1}$, since $s = \texttt{0111}$, these are the indices 2, 3, and 4, so the answer to the query will be $a_2 \oplus a_3 \oplus a_4 = 2 \oplus 3 \oplus 6 = 7$.
4. "1 2 4" — $s = \texttt{0111} \to s = \texttt{0000}$.
5. "2 1" — $s = \texttt{0000}$, there are no indices with $s_i = \texttt{1}$, so since the XOR of an empty set of numbers is considered to be equal to 0, the answer to this query is 0.

[†] A binary string is a string containing only characters 0 or 1.

Input

The first line of the input contains one integer t ($1 \le t \le 10^4$) — the number of test cases in the test.

The descriptions of the test cases follow.

The first line of each test case description contains an integer n ($1 \le n \le 10^5$) — the length of the array.

The second line of the test case contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$).

The third line of the test case contains the binary string s of length n.

The fourth line of the test case contains one integer q ($1 \le q \le 10^5$) — the number of queries.

The subsequent q lines of the test case describe the queries. The first number of each query, $tp \in \{1, 2\}$, characterizes the type of the query: if $tp = 1$, then 2 integers $1 \le l \le r \le n$ follow, meaning that the operation of type 1 should be performed with parameters l, r, and if $tp = 2$, then one integer $g \in \{0, 1\}$ follows, meaning that the operation of type 2 should be performed with parameter g.

It is guaranteed that the sum of n over all test cases does not exceed 10^5, and also that the sum of q over all test cases does not exceed 10^5.

Output

For each test case, and for each query of type 2 in it, output the answer to the corresponding query.

Examples

Input	Output
5 5 1 2 3 4 5 01000 7 2 0 2 1 1 2 4 2 0 2 1 1 1 3 2 1 6 12 12 14 14 5 5 001001 3 2 1 1 2 4 2 1 4 7 7 7 777 1111 3 2 0 1 2 3 2 0 2 1000000000 996179179 11 1 2 1 5 1 42 20 47 7 00011 5 1 3 4 1 1 1 1 3 4 1 2 4 2 0	3 2 6 7 7 11 7 0 0 16430827 47

Note

Let's analyze the first test case:

1. "2 0" — we are interested in the indices i for which $s_i = 0$, since $s = 01000$, these are the indices $1, 3, 4$, and 5, so the answer to the query will be $a_1 \oplus a_3 \oplus a_4 \oplus a_5 = 1 \oplus 3 \oplus 4 \oplus 5 = 3$.
2. "2 1" — we are interested in the indices i for which $s_i = 1$, since $s = 01000$, the only suitable index is 2, so the answer to the query will be $a_2 = 2$.
3. "1 2 4" — we need to replace the characters s_2, s_3, s_4 with their opposites, so before the query $s = 01000$, and after the query: $s = 00110$.

4. "2 0" — we are interested in the indices i for which $s_i = 0$, since $s = 00110$, these are the indices $1, 2$, and 5, so the answer to the query will be $a_1 \oplus a_2 \oplus a_5 = 1 \oplus 2 \oplus 5 = 6$.
5. "2 1" — we are interested in the indices i for which $s_i = 1$, since $s = 00110$, these are the indices 3 and 4, so the answer to the query will be $a_3 \oplus a_4 = 3 \oplus 4 = 7$.
6. "1 1 3" — $s = 00110 \rightarrow s = 11010$.
7. "2 1" — we are interested in the indices i for which $s_i = 1$, since $s = 11010$, these are the indices $1, 2$, and 4, so the answer to the query will be $a_1 \oplus a_2 \oplus a_4 = 1 \oplus 2 \oplus 4 = 7$.